

CPE 241 Database Systems (Module 3)

Mock Final Exam (ข้อสอบจำลองปลายภาค)

เวลาสอบ: 2 ชั่วโมง (13.00–15.00)

คะแนนเต็ม 70 คะแนน

ประกาศและขอบเขตข้อสอบ

ข้อสอบฉบับนี้ออกแบบให้ครอบคลุม Lecture ทั้งหมด โดยเน้น Lecture 9–12 ได้แก่ Query Optimization, Multiuser Environment, NoSQL/MongoDB และ Advanced Topics รวมถึง SQL, Relational Algebra (RA), Tuple Relational Calculus (TRC), Domain Relational Calculus (DRC) และ Normalization

สิ่งที่อนุญาตให้นำเข้าห้องสอบ: เครื่องเขียนและบัตรนักศึกษาเท่านั้น เว้นแต่อาจารย์ผู้คุมสอบประกาศเพิ่มเติม

ลักษณะข้อสอบ: Part 1 Multiple Choice 40 ข้อ, Part 2 Short Answer 10 ข้อ, Part 3 แสดงวิธีทำ 3 ข้อ

Part 1: Multiple Choice Questions (40 Marks)

คำชี้แจง: เลือกคำตอบที่ถูกต้องที่สุด (ข้อละ 1 คะแนน)

- ข้อใดอธิบายลักษณะของ Relational Calculus ได้ถูกต้องที่สุด?
 - เป็นภาษาแบบ Procedural ที่ต้องระบุลำดับการ Join ทุกขั้น
 - เป็นภาษาแบบ Non-procedural ที่ระบุว่าต้องการข้อมูลใดโดยไม่ระบุวิธีดึงข้อมูล
 - ใช้ได้เฉพาะกับฐานข้อมูล NoSQL เท่านั้น
 - ใช้เพื่อสร้าง Index โดยตรงใน DBMS
- ความแตกต่างหลักระหว่าง TRC และ DRC คือข้อใด?
 - TRC ใช้ตัวแปรแทนทั้ง tuple ส่วน DRC ใช้ตัวแปรแทนค่าใน domain หรือ column
 - TRC เป็นภาษา SQL ส่วน DRC เป็นภาษา JavaScript
 - TRC ใช้ได้เฉพาะกับ JOIN ส่วน DRC ใช้ได้เฉพาะกับ GROUP BY
 - TRC เป็น physical plan ส่วน DRC เป็น logical plan
- Relational Algebra จัดเป็นภาษาประเภทใด และเหตุผลคืออะไร?
 - Non-procedural เพราะระบุเฉพาะผลลัพธ์ที่ต้องการ
 - Procedural เพราะระบุขั้นตอนการดำเนินการกับ relation เช่น selection, projection, join
 - Declarative เพราะไม่เกี่ยวข้องกับ execution plan
 - Schema-less เพราะไม่ต้องรู้โครงสร้าง relation

4. ลำดับ Query Execution Process ที่ถูกต้องที่สุดคือข้อใด?

- a. Optimization → Parsing → Execution → Binding → Result
- b. Parsing → Binding → Optimization → Execution → Result Output
- c. Binding → Execution → Parsing → Result Output → Optimization
- d. Execution → Parsing → Optimization → Binding → Result

5. Query Execution Plan โดยทั่วไปไม่รวมข้อมูลประเภทใด?

- a. Access method เช่น Full Table Scan, Index Scan, Index Seek
- b. Join strategy เช่น Nested Loop Join, Hash Join, Merge Join
- c. Cost estimation ของแต่ละ operation
- d. รหัสผ่านของ user ที่รัน query

6. จากสมการด้านล่าง เป็นเทคนิค Query Optimization แบบใด?

$$\pi_{\text{name,grade}}(\sigma_{\text{grade}='A'}(\text{STUDENT} \bowtie \text{GRADE})) \Rightarrow \pi_{\text{name,grade}}(\text{STUDENT} \bowtie \sigma_{\text{grade}='A'}(\text{GRADE}))$$

- a. Predicate Pushdown
- b. Projection Pushdown
- c. Materialized View Rewrite
- d. Hash Partitioning

7. หากตาราง STUDENT มี 120 columns แต่ query ต้องใช้เพียง student_id และ name ก่อนนำไป Join เทคนิคใดควรใช้เพื่อลดข้อมูลเร็วขึ้น?

- a. Projection Pushdown
- b. Dirty Read
- c. Full Outer Join
- d. Data Lakehouse

8. ข้อใดอธิบายเหตุผลที่ควรใช้ WHERE แทน HAVING เมื่อเงื่อนไขนั้นไม่ต้องรอผล aggregate?

- a. WHERE กรองก่อน GROUP BY ทำให้จำนวนแถวที่ต้อง aggregate ลดลง
- b. WHERE ทำงานหลัง HAVING เสมอ
- c. HAVING ใช้กับ aggregate ไม่ได้
- d. WHERE ทำให้ query เป็น NoSQL โดยอัตโนมัติ

9. Index มี trade-off หลักอย่างไร?
- ช่วยให้ SELECT เร็วขึ้น แต่เพิ่มภาระ INSERT, UPDATE, DELETE และใช้พื้นที่เพิ่ม
 - ทำให้ทุก query กลายเป็น O(1) เสมอโดยไม่มีต้นทุน
 - ใช้แทน Primary Key ได้เสมอ
 - ทำให้ตารางเข้าสู่ 3NF โดยอัตโนมัติ
10. Query ต่อไปนี้ควรได้ประโยชน์จาก Index ประเภทใดมากที่สุด?
- ```
SELECT * FROM EMPLOYEE WHERE DeptID = 5 AND Salary > 50000;
```
- Index บน Name เพียงอย่างเดียว
  - Composite index บน (DeptID, Salary)
  - Full-text index บน Address
  - Hash index บน Photo
11. หากต้อง Join ตาราง A 1,000 แถว, B 400,000 แถว และ C 500 แถว heuristic ที่เหมาะสมคือข้อใด?
- Join ตารางใหญ่ที่สุดก่อนเสมอ
  - Join ตารางเล็กก่อน เช่น A กับ C แล้วค่อย Join B
  - ห้ามใช้ Index กับ Join column
  - ใช้ Cartesian Product ก่อนทุกครั้ง
12. ลูกค้านับจำนวนแถวจำนวนมากและต้องกระจายข้อมูลตามช่วงตัวอักษรนามสกุล A-H, I-R, S-Z ควรใช้ partitioning แบบใดตามหลักการ?
- Vertical Partitioning
  - Horizontal Partitioning หรือ Range Sharding
  - Projection Pushdown
  - Full-text Indexing
13. Vertical Partitioning หมายถึงข้อใด?
- แบ่งตารางตามแถว เช่น ภูมิภาคหรือช่วงตัวอักษร
  - แบ่งตารางตาม columns เช่น personal info อยู่ server หนึ่ง และ transaction history อยู่อีก server หนึ่ง
  - สร้างสำเนาข้อมูลทั้งตารางไปทุก node
  - บังคับให้ทุก transaction เป็น Serializable
14. Replication ใน Distributed Database มีประโยชน์หลักข้อใด?
- ลดจำนวนตารางให้เหลือตารางเดียว
  - เพิ่ม reliability, availability และช่วยให้ระบบทนต่อ node failure ได้ดีขึ้น
  - ทำให้ไม่ต้องใช้ concurrency control
  - ทำให้ข้อมูลทั้งหมดกลายเป็น schema-less

15. ในการโอนเงินจากบัญชี A ไป B หากระบบตัดเงินจาก A แล้วดับก่อนเพิ่มเงินให้ B คุณสมบัติ ACID ใดช่วยให้ธุรกรรมต้องสำเร็จทั้งหมดหรือยกเลิกทั้งหมด?
- Atomicity
  - Consistency
  - Isolation
  - Durability
16. Dirty Read หมายถึงข้อใด?
- อ่านข้อมูลที่ transaction อื่นยังไม่ commit และอาจ rollback ได้
  - อ่าน row เดิมสองครั้งแล้วค่าเปลี่ยนเพราะ transaction อื่น update แล้ว commit
  - query รอบที่สองมี row ใหม่เพิ่มเข้ามาตามเงื่อนไขเดิม
  - อ่านข้อมูลจาก Data Lake โดยไม่มี schema
17. Isolation level ใดเป็นระดับต่ำสุดที่ป้องกัน Non-repeatable Read ได้ตามตาราง ANSI SQL ใน lecture?
- Read Uncommitted
  - Read Committed
  - Repeatable Read
  - Serializable
18. Phantom Read ถูกป้องกันอย่างเข้มที่สุดด้วย isolation level ใด?
- Read Uncommitted
  - Read Committed
  - Repeatable Read
  - Serializable
19. เมื่อ MySQL เปิด autocommit = 1 โดย default ผลที่เกิดขึ้นคือข้อใด?
- ทุก SQL statement ถูก commit ทันทีหลังรันสำเร็จ ทำให้ rollback ย้อน statement นั้นไม่ได้
  - ทุก transaction ถูก rollback อัตโนมัติหลัง SELECT
  - DBMS จะปิด Index ทั้งหมด
  - ห้ามใช้ START TRANSACTION
20. คำสั่ง SAVEPOINT ใช้ทำอะไร?
- สร้าง user ใหม่ในระบบ
  - ตั้งจุดภายใน transaction เพื่อให้ rollback กลับไปยังจุดนั้นได้
  - บังคับให้ view เป็น materialized view
  - สร้าง schema สำหรับ MongoDB

21. `SELECT . . . FOR UPDATE` ภายใน transaction มีผลหลักอย่างไร?
- a. อ่านข้อมูลโดยไม่ lock row ใด ๆ
  - b. lock row ที่เลือกไว้เพื่อกัน transaction อื่นมา update จนกว่า commit หรือ rollback
  - c. สร้าง trigger หลังอ่านข้อมูล
  - d. แปลง query ให้เป็น DRC
22. Trigger แบบ `BEFORE INSERT` เหมาะกับงานใดมากที่สุด?
- a. Validation หรือปรับค่า `NEW` ก่อน row ถูก insert จริง
  - b. สร้าง backup ของทั้ง database หลัง commit
  - c. ปิด isolation level
  - d. ทำ schema-on-read
23. ข้อใดเป็นข้อจำกัดของ Trigger ใน MySQL ตาม lecture?
- a. MySQL รองรับเฉพาะ row-level trigger ไม่รองรับ statement-level trigger
  - b. Trigger ต้องเขียนด้วย Python เท่านั้น
  - c. Trigger สามารถเรียก `COMMIT` และ `ROLLBACK` ได้เสมอ
  - d. ตารางหนึ่งต้องมี trigger อย่างน้อย 5 ตัว
24. Assertion ต่างจาก Check Constraint อย่างไรในภาพรวม?
- a. Assertion สามารถนิยามเงื่อนไขข้ามหลายตารางได้ แต่หลาย DBMS ไม่รองรับโดยตรง
  - b. Check Constraint ใช้ได้เฉพาะกับ MongoDB
  - c. Assertion ใช้เพื่อสร้าง user account เท่านั้น
  - d. ทั้งสองอย่างคือคำสั่งเดียวกันทุกประการ
25. ต้องการให้ user `student` อ่านและเพิ่มข้อมูลในตาราง `mydb.employee` ได้ ควรใช้คำสั่งใด?
- a. `GRANT SELECT, INSERT ON mydb.employee TO 'student'@'localhost';`
  - b. `REVOKE SELECT, INSERT ON mydb.employee FROM 'student'@'localhost';`
  - c. `DROP USER 'student'@'localhost';`
  - d. `ROLLBACK TO student;`
26. View ใน SQL คืออะไร?
- a. Virtual table จากผลลัพธ์ query ซึ่งโดยทั่วไปไม่เก็บข้อมูลเอง
  - b. Physical table ที่มีข้อมูลซ้ำเต็มชุดเสมอ
  - c. Index ชนิดหนึ่งที่ใช้แทน Primary Key
  - d. Isolation level แบบหนึ่ง

27. เงื่อนไขใดทำให้ View มีโอกาสเป็น updatable view ใน MySQL มากที่สุด?
- View อิงจากตารางเดียว และไม่มี DISTINCT, GROUP BY, aggregate หรือ join ซับซ้อน
  - View ต้องมี UNION อย่างน้อยหนึ่งครั้ง
  - View ต้องซ่อน Primary Key ทุกครั้ง
  - View ต้อง Join อย่างน้อยสามตาราง
28. NoSQL หมายถึงอะไรในบริบทของ lecture?
- Not Only SQL เป็นกลุ่มฐานข้อมูล non-relational ที่เหมาะกับงานบางประเภท
  - No Security Query Language
  - ฐานข้อมูลที่ห้ามใช้ JSON
  - ระบบสำรองข้อมูลของ RDBMS เท่านั้น
29. ลำดับโครงสร้างของ MongoDB จากใหญ่ไปเล็กคือข้อใด?
- Database → Collection → Document → Field
  - Field → Document → Database → Collection
  - Table → Row → Column → Index
  - Node → Edge → Graph → Database
30. MongoDB Document มีลักษณะตรงกับข้อใด?
- Stored in BSON, รองรับ nested object, array และ field ที่ยืดหยุ่น
  - ต้องมี schema เดียวกันทุก document แบบเข้มงวดเหมือน table ใน SQL
  - เก็บข้อมูลได้เฉพาะ string เท่านั้น
  - ห้ามมีค่า null และ Date
31. NoSQL แบบ Key-Value เหมาะกับงานใดมากที่สุด?
- Caching, session storage และ real-time counters
  - ออกแบบ ERD แบบ normalized ถึง BCNF
  - แปลง TRC เป็น DRC
  - สร้าง OLAP cube ที่มี schemaตายตัว
32. Graph Database เช่น Neo4j เหมาะกับโจทย์ใด?
- Social network และ recommendation ที่เน้นความสัมพันธ์ระหว่าง entities
  - Transaction bank transfer ที่ต้อง normalized สูงเท่านั้น
  - เก็บไฟล์ raw image โดยไม่วิเคราะห์
  - สร้าง trigger ใน MySQL

33. Mongoose ใน Node.js มีบทบาทอะไรกับ MongoDB?

- a. เป็น ODM ที่ช่วย model schema, validation, CRUD และ query building
- b. เป็น isolation level ของ MySQL
- c. เป็นชนิดของ Index ใน Relational DB
- d. เป็นคำสั่ง SQL สำหรับสร้าง view

34. ข้อใดเป็น JSON ที่ถูกต้องตามกฎพื้นฐาน?

- a. { "name": "Alice", "age": 21, "isStudent": true }
- b. { 'name': 'Alice', age: 21, }
- c. { name = "Alice"; age = 21 }
- d. [name: Alice, age: 21]

35. ข้อใดจับคู่ OLTP และ OLAP ได้ถูกต้องที่สุด?

- a. OLTP: short frequent transactions; OLAP: complex analytical read-heavy queries
- b. OLTP: historical cube only; OLAP: bank transfer real-time only
- c. OLTP: schema-on-read; OLAP: no aggregation
- d. OLTP และ OLAP เหมือนกันทุกด้าน

36. Data Warehouse มีลักษณะเด่นข้อใด?

- a. Structured, cleaned, historical data ผ่าน ETL และ schema-on-write เพื่อ analytical queries
- b. Raw unprocessed data ทุกชนิดโดยไม่กำหนด schema จนกว่าจะอ่าน
- c. ใช้แทน transaction database เพื่อทำ bank transfer real-time
- d. ห้ามเก็บข้อมูลย้อนหลัง

37. Data Lake แตกต่างจาก Data Warehouse อย่างไร?

- a. Data Lake เก็บ raw/unprocessed data และใช้ schema-on-read ส่วน Data Warehouse เก็บข้อมูลที่จัดโครงสร้างและทำความสะอาดแล้ว
- b. Data Lake ใช้ได้เฉพาะกับ OLTP เท่านั้น
- c. Data Warehouse ไม่มี schema
- d. ทั้งสองอย่างใช้กับข้อมูลชนิดเดียวกันและไม่มีความต่าง

38. บริษัทเก็บ log clickstream, รูปภาพ, JSON event และไฟล์ raw จำนวนมากไว้ก่อน โดยยังไม่รู้ว่าจะวิเคราะห์อะไรในอนาคต ควรเลือกอะไรเหมาะสมสุด?

- a. Transaction Database
- b. Data Warehouse
- c. Data Lake
- d. Updatable View

39. ใน Distributed Database เมื่อเกิด network failure มักต้องพิจารณา trade-off ใด?

- a. Consistency, Availability และ Partition Tolerance
- b. Projection, Selection และ Rename
- c. Atomicity, Indexing และ Mongoose
- d. JSON, BSON และ XML

40. Elasticity ใน Cloud Computing หมายถึงข้อใด?

- a. ความสามารถในการเพิ่มหรือลด resource ตาม demand อัตโนมัติหรือรวดเร็ว
- b. การทำให้ข้อมูลทุกตัวเป็น atomic value
- c. การซ่อน column ด้วย view
- d. การ rollback transaction ไปยัง savepoint



## Part 2: Short Answer Questions (10 Marks)

คำชี้แจง: ตอบสั้น กระชับ และตรงประเด็น (ข้อละ 1 คะแนน)

1. อธิบายความต่างระหว่าง TRC และ DRC ในแง่ของตัวแปรที่ใช้ในนิพจน์ relational calculus
2. จาก query `SELECT * FROM STUDENT WHERE program='CPE';` จงบอกเหตุผล 1 ข้อว่าทำไม `SELECT *` อาจทำให้ performance แย่กว่าการเลือกเฉพาะ column ที่ใช้จริง
3. Index Scan กับ Index Seek ต่างกันอย่างไรในภาพรวม?
4. ใน transaction isolation จงยกตัวอย่างสั้น ๆ ของ Phantom Read
5. ถ้าต้องการ undo เฉพาะบางคำสั่งภายใน transaction โดยไม่ทิ้ง transaction ทั้งหมด ควรใช้คำสั่งใดร่วมกับ `ROLLBACK TO?`
6. Trigger แบบ `AFTER INSERT` เหมาะกับงานประเภท logging/auditing อย่างไร?
7. View ช่วยเรื่อง security ได้อย่างไร จงตอบด้วยตัวอย่างสั้น ๆ
8. เขียนลำดับชั้นของ MongoDB ตั้งแต่ Database ลงไปถึง Field
9. เลือกคำตอบระหว่าง Database, Data Warehouse, Data Lake: ระบบ dashboard ผู้บริหารที่วิเคราะห์ยอดขายย้อนหลังหลายปีจากข้อมูลที่ clean แล้วควรใช้อะไร?
10. ตาราง `ENROLLMENT(Sid, CourseId, Sname, CourseName, Grade)` มีคีย์ผสม `{Sid, CourseId}` และมี `Sid → Sname` กับ `CourseId → CourseName` ขัดกับ Normal Form ระดับใดเป็นหลัก?

### Part 3: Long Questions (20 Marks)

คำชี้แจง: แสดงวิธีทำให้ครบถ้วน ข้อ 1-2 ให้เติมช่องว่างของ SQL, RA, TRC และ DRC ให้สมบูรณ์ ส่วนข้อ 3 ให้ทำ Dependency/Normalization

#### Question 1: Query Writing 4 Forms (7 Marks)

กำหนด schema ของระบบร้านค้าออนไลน์ดังนี้

- CUSTOMER(Cid, Cname, City, Tier)
- ORDERS(Oid, Cid, OrderDate, Status)
- ORDER\_ITEM(Oid, Pid, Qty, UnitPrice)
- PRODUCT(Pid, Pname, Category, SupplierId)
- SUPPLIER(SupplierId, Sname, Country)

โจทย์: หาชื่อและ Tier ของลูกค้าที่อยู่เมือง 'Bangkok' และเคยซื้อสินค้า Category = 'Storage' จาก supplier ประเทศ 'Japan' โดย order ต้องมี Status = 'PAID'

หมายเหตุ: ต้องคืนผลลัพธ์เป็น Cname, Tier เท่านั้น

##### 1.1 SQL (2 คะแนน)

```
1 SELECT DISTINCT C.Cname, C.Tier
2 FROM CUSTOMER C
3 JOIN ORDERS O ON
4 JOIN ORDER_ITEM I ON
5 JOIN PRODUCT P ON
6 JOIN SUPPLIER S ON
7 WHERE C.City = 'Bangkok'
8 AND O.Status = 'PAID'
9 AND P.Category = 'Storage'
10 AND ;
```

##### 1.2 Relational Algebra (2 คะแนน)

$$\pi_{\text{Cname, Tier}} \left( \sigma_{\left( \begin{array}{l} \text{CUSTOMER} \bowtie \text{ORDERS} \bowtie \text{ORDER\_ITEM} \\ \bowtie \text{PRODUCT} \bowtie \text{SUPPLIER} \end{array} \right)} \right)$$

##### 1.3 TRC (1.5 คะแนน)

$$\{ c.\text{Cname}, c.\text{Tier} \mid c \in \text{CUSTOMER} \wedge c.\text{City} = 'Bangkok' \wedge \\ \exists o \in \text{ORDERS} \exists i \in \text{ORDER\_ITEM} \exists p \in \text{PRODUCT} \exists s \in \text{SUPPLIER} \\ ( \quad ) \}$$

#### 1.4 DRC (1.5 คะแนน)

$$\{ cn, tier \mid \exists cid, city, oid, odate, status, pid, qty, price, pn, cat, sid, sn, country$$
$$(CUSTOMER(cid, cn, city, tier) \wedge ORDERS(oid, cid, odate, status) \wedge$$
$$ORDER\_ITEM(oid, pid, qty, price) \wedge PRODUCT(pid, pn, cat, sid) \wedge$$
$$SUPPLIER(sid, sn, country) \wedge \boxed{\phantom{ORDER\_ITEM(oid, pid, qty, price)}} ) \}$$

## Question 2: Universal Query / Division (7 Marks)

กำหนด schema ของระบบลงทะเบียนเรียนดังนี้

- STUDENT(Sid, Sname, Major, Year)
- COURSE(Cid, Cname, Track)
- ENROLL(Sid, Cid, Grade)

โจทย์: ทหารหัสและชื่อนักศึกษา Major = 'CPE' ที่ลงครบทุกวิชาใน Track = 'Database' และทุกวิชานั้นได้ Grade อยู่ใน { 'A', 'B+', 'B' } หรือดีกว่าเทียบกับตามโจทย์นี้

หมายเหตุ: ให้ถือว่า 'A', 'B+', 'B' เป็นกลุ่มเกรดผ่านเงื่อนไข

### 2.1 SQL แบบ NOT EXISTS ซ้อนกัน (2 คะแนน)

```
1 SELECT S.Sid, S.Sname
2 FROM STUDENT S
3 WHERE S.Major = 'CPE'
4 AND NOT EXISTS (
5 SELECT 1
6 FROM COURSE C
7 WHERE C.Track = 'Database'
8 AND NOT EXISTS (
9 SELECT 1
10 FROM ENROLL E
11 WHERE
12 AND
13 AND
14)
15);
```

### 2.2 Relational Algebra แบบ Division (2 คะแนน)

$$\begin{aligned} DBCourse &= \pi_{Cid}(\sigma_{Track='Database'}(COURSE)) \\ GoodEnroll &= \pi_{Sid,Cid}(\sigma_{\text{}(ENROLL)) \\ GoodStudent &= \text{ \\ Answer &= \pi_{Sid,Sname}(\sigma_{Major='CPE'}(STUDENT) \bowtie \text{ GoodStudent) \end{aligned}$$

### 2.3 TRC (1.5 คะแนน)

$$\begin{aligned} &\{ s.Sid, s.Sname \mid s \in STUDENT \wedge s.Major = 'CPE' \wedge \\ &\quad \forall c \in COURSE (c.Track = 'Database' \Rightarrow \\ &\quad \exists e \in ENROLL ( \text{ )) \} \end{aligned}$$

#### 2.4 DRC (1.5 คะแนน)

$$\{ sid, sname \mid \exists major, yr (STUDENT(sid, sname, major, yr) \wedge major = 'CPE' \wedge \\ \forall cid, cname, track (COURSE(cid, cname, track) \wedge track = 'Database' \Rightarrow \\ \exists grade (ENROLL(sid, cid, grade) \wedge \boxed{\phantom{grade}})) ) \}$$

### Question 3: Functional Dependency and Normalization (6 Marks)

กำหนด relation สำหรับบันทึกงานโปรเจกต์ดังนี้

```
PROJECT_ASSIGNMENT(EmpID, ProjectID, EmpName, DeptID, DeptName,
 ProjectName, ClientID, ClientName,
 ManagerID, ManagerName, Hours, SkillLevel)
```

กำหนด Primary Key เป็น {EmpID, ProjectID} และมี Functional Dependencies:

- FD1: EmpID  $\rightarrow$  EmpName, DeptID
- FD2: DeptID  $\rightarrow$  DeptName
- FD3: ProjectID  $\rightarrow$  ProjectName, ClientID, ManagerID
- FD4: ClientID  $\rightarrow$  ClientName
- FD5: ManagerID  $\rightarrow$  ManagerName
- FD6: {EmpID, ProjectID}  $\rightarrow$  Hours, SkillLevel

3.1 (1 คะแนน) Relation เดิมอยู่ใน Normal Form สูงสุดระดับใด และขัดกับกฎใดของระดับถัดไป? ให้ระบุ dependency ที่เป็นปัญหา

3.2 (2 คะแนน) แยก relation ให้ผ่าน 2NF โดยระบุ schema, Primary Key และ Foreign Key

3.3 (3 คะแนน) แยกต่อให้ผ่าน 3NF โดยระบุ schema สุดท้ายทั้งหมด, Primary Key และ Foreign Key พร้อมอธิบายว่าแก้ transitive dependency ตัวใดบ้าง

## เฉลยและแนวคิด (Answer Key)

ส่วนนี้สำหรับตรวจหลังทำข้อสอบ แนะนำให้ทำข้อสอบเต็มเวลาก่อนเปิดเฉลย

### Part 1: Multiple Choice Questions

1. b. Relational Calculus เป็น Non-procedural
2. a. TRC ใช้ tuple variables, DRC ใช้ domain variables
3. b. RA เป็น Procedural
4. b. Parsing, Binding, Optimization, Execution, Result
5. d. Query plan ไม่แสดง password
6. a. Predicate Pushdown
7. a. Projection Pushdown
8. a. WHERE กรองก่อน aggregate
9. a. Index ช่วยอ่านแต่มีต้นทุนเขียนและ storage
10. b. Composite index บน (DeptID, Salary)
11. b. Join ตารางเล็กก่อนตาม heuristic
12. b. แบ่งตามแถวคือ Horizontal หรือ Range Sharding
13. b. Vertical คือแบ่งตาม columns
14. b. Replication ช่วย reliability และ fault tolerance
15. a. Atomicity
16. a. Dirty Read คืออ่านค่าที่ยังไม่ commit
17. c. Repeatable Read
18. d. Serializable
19. a. Autocommit commit ทุก statement
20. b. SAVEPOINT ตั้งจุด rollback ภายใน transaction
21. b. SELECT FOR UPDATE lock row
22. a. BEFORE INSERT ใช้ validate/modify ก่อน insert
23. a. MySQL มี row-level trigger
24. a. Assertion ใช้เงื่อนไขข้าม table ได้ แต่หลาย DBMS ไม่รองรับ
25. a. GRANT SELECT, INSERT
26. a. View เป็น virtual table
27. a. Updatable view มักต้องอิง table เดียวและไม่ aggregate/join ซับซ้อน
28. a. NoSQL = Not Only SQL
29. a. Database, Collection, Document, Field
30. a. MongoDB เก็บ BSON และ nested/array ได้
31. a. Key-value เหมาะกับ caching/session
32. a. Graph เหมาะกับ relationship-heavy use cases
33. a. Mongoose เป็น ODM
34. a. JSON ต้องใช้ double quotes และไม่มี trailing comma
35. a. OLTP transaction สั้นถี่, OLAP analytical read-heavy
36. a. Data Warehouse คือ structured historical analytical store
37. a. Data Lake เป็น raw/schema-on-read
38. c. raw diverse future analytics คือ Data Lake
39. a. CAP trade-off
40. a. Elasticity คือเพิ่มลด resource ตาม demand

## Part 2: Short Answer Questions

1. TRC ใช้ตัวแปรแทนทั้ง tuple/row เช่น `t.Attribute`; DRC ใช้ตัวแปรแทนค่าในแต่ละ domain/column
2. `SELECT *` ดึง column เกินจำเป็น ทำให้ data transfer สูงขึ้น และลดโอกาสทำ projection pushdown ได้ดี
3. Index Scan ไล่อ่านช่วง/ส่วนของ index หลาย entry; Index Seek กระโดดไปยัง entry ที่เกี่ยวข้องโดยตรงมากกว่า จึงมักมีต้นทุนต่ำกว่า
4. ตัวอย่าง: query เดิมสองรอบใน transaction เดียวกัน

```
SELECT * FROM orders WHERE price > 1000;
```

รอบสองเจอ row ใหม่ที่ transaction อื่น insert และ commit เข้ามา

5. ใช้ SAVEPOINT เช่น `SAVEPOINT sp1;` แล้ว `ROLLBACK TO sp1;`
6. ใช้ insert record ลงตาราง log หลัง row ใหม่ถูก insert สำเร็จ เช่น เก็บ `emp_id`, action, timestamp
7. สร้าง view ที่เลือกเฉพาะ column ปลอดภัย เช่น `id, name, position` แล้ว grant ให้ user โดยไม่ให้เห็น `salary`
8. Database → Collection → Document → Field
9. Data Warehouse
10. ขัดกับ 2NF เพราะมี Partial Dependency จากส่วนหนึ่งของ composite key ไปยัง non-prime attributes



## Part 3: Long Questions

### Question 1

SQL

```
1 SELECT DISTINCT C.Cname, C.Tier
2 FROM CUSTOMER C
3 JOIN ORDERS O ON C.Cid = O.Cid
4 JOIN ORDER_ITEM I ON O.Oid = I.Oid
5 JOIN PRODUCT P ON I.Pid = P.Pid
6 JOIN SUPPLIER S ON P.SupplierId = S.SupplierId
7 WHERE C.City = 'Bangkok'
8 AND O.Status = 'PAID'
9 AND P.Category = 'Storage'
10 AND S.Country = 'Japan';
```

RA

$$\pi_{\text{Cname}, \text{Tier}} \left( \sigma_{\text{City}='Bangkok' \wedge \text{Status}='PAID' \wedge \text{Category}='Storage' \wedge \text{Country}='Japan'} \left( \begin{aligned} &\text{CUSTOMER} \bowtie_{\text{CUSTOMER.Cid}=\text{ORDERS.Cid}} \text{ORDERS} \bowtie_{\text{ORDERS.Oid}=\text{ORDER\_ITEM.Oid}} \text{ORDER\_ITEM} \\ &\bowtie_{\text{ORDER\_ITEM.Pid}=\text{PRODUCT.Pid}} \text{PRODUCT} \bowtie_{\text{PRODUCT.SupplierId}=\text{SUPPLIER.SupplierId}} \text{SUPPLIER} \end{aligned} \right) \right)$$

TRC

$$\{ c.\text{Cname}, c.\text{Tier} \mid c \in \text{CUSTOMER} \wedge c.\text{City} = 'Bangkok' \wedge \\ \exists o \in \text{ORDERS} \exists i \in \text{ORDER\_ITEM} \exists p \in \text{PRODUCT} \exists s \in \text{SUPPLIER} \\ (o.\text{Cid} = c.\text{Cid} \wedge i.\text{Oid} = o.\text{Oid} \wedge p.\text{Pid} = i.\text{Pid} \wedge \\ s.\text{SupplierId} = p.\text{SupplierId} \wedge o.\text{Status} = 'PAID' \wedge \\ p.\text{Category} = 'Storage' \wedge s.\text{Country} = 'Japan') \}$$

DRC

$$\{ cn, tier \mid \exists cid, city, oid, odate, status, pid, qty, price, pn, cat, sid, sn, country \\ (\text{CUSTOMER}(cid, cn, city, tier) \wedge \text{ORDERS}(oid, cid, odate, status) \wedge \\ \text{ORDER\_ITEM}(oid, pid, qty, price) \wedge \text{PRODUCT}(pid, pn, cat, sid) \wedge \\ \text{SUPPLIER}(sid, sn, country) \wedge city = 'Bangkok' \wedge status = 'PAID' \wedge \\ cat = 'Storage' \wedge country = 'Japan') \}$$

### Question 2

SQL

```
1 SELECT S.Sid, S.Sname
2 FROM STUDENT S
3 WHERE S.Major = 'CPE'
4 AND NOT EXISTS (
5 SELECT 1
6 FROM COURSE C
7 WHERE C.Track = 'Database'
8 AND NOT EXISTS (
9 SELECT 1
```

```

10 FROM ENROLL E
11 WHERE E.Sid = S.Sid
12 AND E.Cid = C.Cid
13 AND E.Grade IN ('A', 'B+', 'B')
14)
15);

```

RA

$$\begin{aligned}
DBCOURSE &= \pi_{Cid}(\sigma_{Track='Database'}(COURSE)) \\
GoodEnroll &= \pi_{Sid,Cid}(\sigma_{Grade \in \{'A', 'B+', 'B'\}}(ENROLL)) \\
GoodStudent &= GoodEnroll \div DBCourse \\
Answer &= \pi_{Sid,Sname}(\sigma_{Major='CPE'}(STUDENT) \bowtie_{STUDENT.Sid=GoodStudent.Sid} GoodStudent)
\end{aligned}$$

TRC

$$\begin{aligned}
&\{ s.Sid, s.Sname \mid s \in STUDENT \wedge s.Major = 'CPE' \wedge \\
&\quad \forall c \in COURSE (c.Track = 'Database' \Rightarrow \\
&\quad \exists e \in ENROLL (e.Sid = s.Sid \wedge e.Cid = c.Cid \wedge e.Grade \in \{'A', 'B+', 'B'\})) \}
\end{aligned}$$

DRC

$$\begin{aligned}
&\{ sid, sname \mid \exists major, yr (STUDENT(sid, sname, major, yr) \wedge major = 'CPE' \wedge \\
&\quad \forall cid, cname, track (COURSE(cid, cname, track) \wedge track = 'Database' \Rightarrow \\
&\quad \exists grade (ENROLL(sid, cid, grade) \wedge grade \in \{'A', 'B+', 'B'\}))) \}
\end{aligned}$$

### Question 3: Normalization เฉลยละเอียดพร้อม FD Diagram

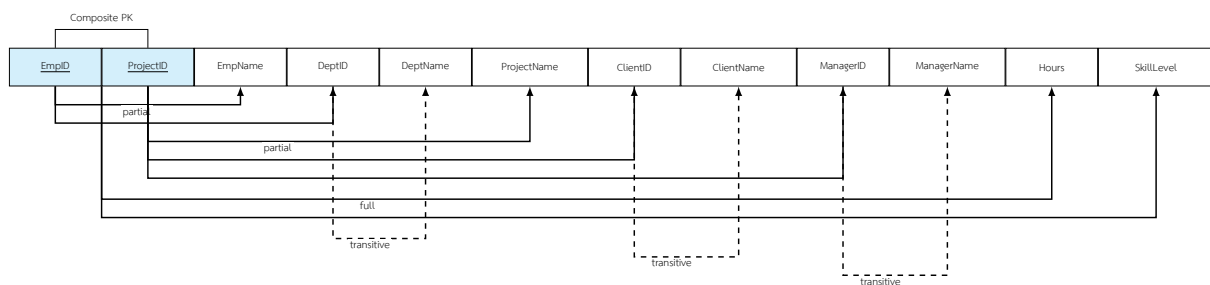
เริ่มจาก Relation เดิม

PROJECT\_ASSIGNMENT(EmpID, ProjectID, EmpName, DeptID, DeptName, ProjectName, ClientID, ClientName, ManagerID, ManagerName, Hours, SkillLevel)

PK = {EmpID, ProjectID}

#### 3.1 ตรวจ Normal Form เดิม

Relation เดิมถือว่าอยู่ใน 1NF เท่านั้น เพราะทุก attribute เป็น atomic value แล้ว แต่ **ไม่ผ่าน 2NF** เนื่องจากมี partial dependency จากบางส่วนของ composite key ไปยัง non-prime attributes



สรุป 3.1:

- Partial Dependency:  $\text{EmpID} \rightarrow \text{EmpName}, \text{DeptID}$
- Partial Dependency:  $\text{ProjectID} \rightarrow \text{ProjectName}, \text{ClientID}, \text{ManagerID}$
- Full Dependency ที่ถูกต้องกับ key ทั้งก้อน:  $\{\text{EmpID}, \text{ProjectID}\} \rightarrow \text{Hours}, \text{SkillLevel}$
- Transitive Dependency ที่จะต้องแก้ใน 3NF:  $\text{DeptID} \rightarrow \text{DeptName}, \text{ClientID} \rightarrow \text{ClientName}, \text{ManagerID} \rightarrow \text{ManagerName}$

#### 3.2 แก้เป็น 2NF

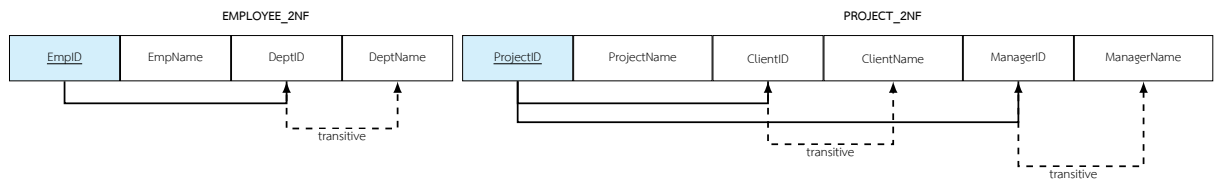
แยก attribute ที่ขึ้นกับ EmpID อย่างเดียว และ attribute ที่ขึ้นกับ ProjectID อย่างเดียว ออกจากตารางกลาง เหลือเฉพาะ attribute ที่ขึ้นกับ key ทั้งก้อนใน ASSIGNMENT

| EMPLOYEE_2NF |         |        |          |
|--------------|---------|--------|----------|
| <u>EmpID</u> | EmpName | DeptID | DeptName |

| PROJECT_2NF      |             |          |            |           |             |
|------------------|-------------|----------|------------|-----------|-------------|
| <u>ProjectID</u> | ProjectName | ClientID | ClientName | ManagerID | ManagerName |

| ASSIGNMENT |                |       |            |
|------------|----------------|-------|------------|
| EmpID (FK) | ProjectID (FK) | Hours | SkillLevel |

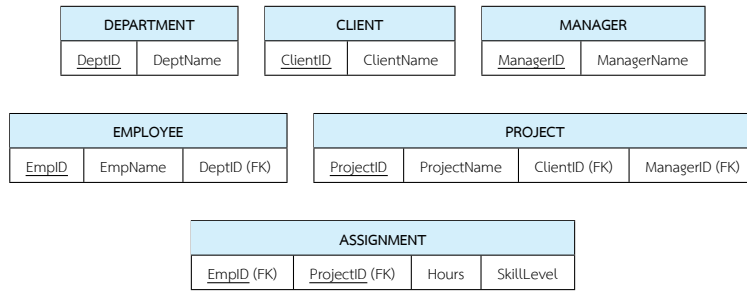
FD diagram หลังได้ 2NF แต่ยังเห็นปัญหา 3NF:



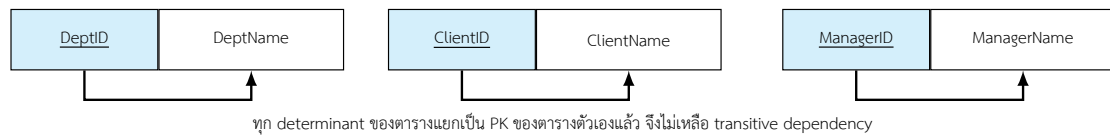
**สรุป 3.2:** ผลลัพธ์ 2NF ถูกต้อง แต่ยังไม่ผ่าน 3NF เพราะยังเก็บ DeptName, ClientName, ManagerName ไว้ในตารางที่ determinant ของมันไม่ใช่ key

### 3.3 แก้เป็น 3NF

แยก transitive dependency ออกเป็นตารางใหม่ โดยให้ determinant ที่เป็น non-key กลายเป็น PK ของตารางใหม่



FD diagram หลัง 3NF:



Mapping FK:

- EMPLOYEE.DeptID อ้างถึง DEPARTMENT.DeptID
- PROJECT.ClientID อ้างถึง CLIENT.ClientID
- PROJECT.ManagerID อ้างถึง MANAGER.ManagerID
- ASSIGNMENT.EmpID อ้างถึง EMPLOYEE.EmpID
- ASSIGNMENT.ProjectID อ้างถึง PROJECT.ProjectID

ประโยคสรุปแบบส่งข้อสอบ:

Relation เดิมอยู่ใน 1NF แต่ไม่ผ่าน 2NF เพราะมี partial dependency จาก EmpID และ ProjectID ซึ่งเป็นเพียงบางส่วนของ composite PK หลังแยกเป็น 2NF แล้วยังไม่ผ่าน 3NF เพราะมี transitive dependency ได้แก่

- EmpID → DeptID → DeptName
- ProjectID → ClientID → ClientName
- ProjectID → ManagerID → ManagerName

ดังนั้นจึงต้องแยก DEPARTMENT, CLIENT, และ MANAGER ออกมา

## RECAP: สรุปเนื้อหาจาก Mock Exam สำหรับอ่านก่อนสอบ

ส่วนนี้สรุปเป็น pattern จำสอบ ไม่ใช่การคัดใจหยาบมาแปะตรง ๆ

### RECAP 1: Query Languages, RA, TRC, DRC

- **Relational Algebra (RA)** เป็นภาษาแบบ Procedural เพราะต้องบอกขั้นตอนว่าจะ select, project, join อย่างไร
- **SQL, TRC, DRC** เป็นภาษาแบบ Non-procedural เพราะบอกเงื่อนไขของข้อมูลที่ต้องการมากกว่าบอกแผนทำงานทีละขั้น
- **TRC** ใช้ตัวแปรแทน tuple ทั้งหมด เช่น  $t \in \text{EmpID}$ ; **DRC** ใช้ตัวแปรแทนค่าในแต่ละ domain หรือแต่ละ column
- เจอคำว่า “มีอย่างน้อยหนึ่ง” ให้นึกถึง  $\exists$  หรือ EXISTS; เจอคำว่า “ทุก” ให้นึกถึง  $\forall$  หรือ pattern แบบ double NOT EXISTS
- Query แนว division เช่น “ลูกค้าซื้อสินค้าทุกชิ้น” หรือ “นักเรียนลงทุกวิชา” มักเขียนด้วยเงื่อนไขว่าไม่มี item ใดที่ขาดไป
- Cartesian Product ได้จำนวนแถวเยอะมากเพราะเป็นการจับคู่ทุกแถวของตารางหนึ่งกับทุกแถวของอีกตารางหนึ่ง จึงควรตามด้วยเงื่อนไข join ที่ถูกต้องเสมอ

### RECAP 2: Query Optimization

- **Predicate Pushdown** คือดันเงื่อนไขกรองแถวให้ทำเร็วที่สุดก่อน join เพื่อลดจำนวน tuple ที่ต้องประมวลผล
- **Projection Pushdown** คือเลือกเฉพาะ column ที่จำเป็นให้เร็วที่สุด เพื่อลด data transfer และลดขนาด intermediate result
- หลีกเลี่ยง SELECT \* เพราะดึง column เกินจำเป็น ทำให้ optimizer มีพื้นที่ optimize น้อยลง และใช้ I/O มากขึ้น
- **WHERE** กรองก่อน **GROUP BY**; **HAVING** กรองหลัง aggregate แล้ว ถ้าเงื่อนไขไม่เกี่ยวกับ aggregate ควรใส่ใน WHERE
- Index ช่วยอ่านเร็วขึ้น แต่ทำให้ INSERT, UPDATE, DELETE ช้าลง เพราะ index structure ต้องถูกดูแลเพิ่ม
- Composite index ต้องคิดลำดับ column ให้ตรงกับเงื่อนไขค้นหาหลัก เช่น ค้นด้วย department ก่อน salary ก็ควรมี index เริ่มจาก department
- Join order มีผลมาก ตารางที่ถูกกรองแล้วเหลือแถวน้อยควรถูก join เร็ว เพื่อไม่ให้ intermediate result โตเกินจำเป็น
- **Horizontal partitioning** แบ่งตามแถว เช่น แบ่ง customer ตามช่วง A-Z หรือแบ่ง order ตามปี; **Vertical partitioning** แบ่งตาม column
- **Replication** คือทำสำเนาข้อมูลหลายชุดเพื่อ reliability และ availability แต่ต้องระวัง consistency และต้นทุน sync

### RECAP 3: Transaction, ACID, Concurrency

- **Atomicity** คือ transaction ต้องสำเร็จทั้งก้อนหรือยกเลิกทั้งก้อน ไม่มีครึ่ง ๆ กลาง ๆ
- **Consistency** คือก่อนและหลัง transaction ฐานข้อมูลต้องไม่ละเมิด constraint
- **Isolation** คือ transaction ที่รันพร้อมกันต้องไม่รบกวนกันจนได้ผลผิดจากที่ควร
- **Durability** คือ commit แล้วข้อมูลต้องอยู่รอดแม้ระบบล่ม

- **Dirty Read** คืออ่านค่าที่ยังไม่ commit; **Non-repeatable Read** คืออ่าน row เดิมสองครั้งแล้วค่าเปลี่ยน; **Phantom Read** คือ query เจียนไขเดิมแล้วรอบหลังมีแถวใหม่โผล่
- Isolation ยิ่งสูงยิ่งกันปัญหา concurrency ได้ดี แต่แลกกับ performance และโอกาส lock/block ที่มากขึ้น
- **Serializable** ปลอดภัยสุดในเชิง isolation เพราะทำให้ผลลัพธ์เหมือน transaction รันเรียงทีละตัว
- **SAVEPOINT** ใช้ตั้งจุด rollback ย่อยใน transaction; **SELECT FOR UPDATE** ใช้ lock row ที่กำลังจะอ่านเพื่อแก้ไข
- **Autocommit** ทำให้แต่ละ SQL statement ถูก commit อัตโนมัติ จึงต้องปิดหรือเริ่ม transaction เองถ้าต้องการ rollback หลาย statement พร้อมกัน

#### RECAP 4: Advanced SQL, Trigger, View, Security

- **Trigger** คือ logic ที่ทำงานอัตโนมัติเมื่อเกิด event เช่น INSERT, UPDATE, DELETE
- **BEFORE Trigger** เหมาะกับการ validate หรือปรับค่าก่อนบันทึก; **AFTER Trigger** เหมาะกับ audit log หรือ action หลังบันทึกสำเร็จ
- **Assertion** ใช้ตรวจ constraint ที่อาจเกี่ยวหลายตาราง แต่ DBMS จำนวนมากไม่รองรับเต็มรูปแบบ จึงมักต้องใช้ trigger หรือ application logic แทน
- **GRANT** ให้สิทธิ์, **REVOKE** ถอนสิทธิ์; เวลาให้สิทธิ์ควรให้เท่าที่จำเป็น ไม่ควรให้กว้างเกินงาน
- **View** เป็น virtual table ใช้ซ่อน column สำคัญ เช่น salary หรือใช้ทำ interface ที่อ่านง่ายให้ user
- View ที่ update กลับตารางจริงได้มักต้องเรียบง่าย อิง table เดียว และไม่ควรมี aggregate, GROUP BY, DISTINCT, หรือ join ซับซ้อน

#### RECAP 5: NoSQL and MongoDB

- **NoSQL** แปลว่า Not Only SQL เหมาะกับงานที่ schema ยืดหยุ่น scale ง่าย หรือข้อมูลมีโครงสร้างหลากหลาย
- โครงสร้าง MongoDB จำจากใหญ่ไปเล็ก: **Database** → **Collection** → **Document** → **Field**
- MongoDB เก็บข้อมูลแบบ BSON ซึ่งคล้าย JSON แต่รองรับชนิดข้อมูลเพิ่มเติมและเหมาะกับการจัดเก็บจริงใน database
- JSON ต้องใช้ double quotes กับ key/string และห้ามมี trailing comma
- Document database เหมาะกับข้อมูลที่มี nested object หรือ array; key-value เหมาะกับ cache/session; graph เหมาะกับข้อมูล relationship-heavy
- **Mongoose** เป็น ODM ฝั่ง Node.js ใช้กำหนด schema/model และช่วยคุยกับ MongoDB ในระดับ object
- จุดที่ออกสอบบ่อยคือให้เลือกว่า use case แบบนี้ควรใช้ database ประเภทใด ไม่ใช่จำชื่ออย่างเดียว

## RECAP 6: OLTP, OLAP, Warehouse, Lake, Cloud

- OLTP เน้น transaction สั้น ๆ จำนวนมาก เช่น insert order, update stock, transfer money
- OLAP เน้น query วิเคราะห์ข้อมูลก้อนใหญ่ เช่น report, dashboard, trend, aggregation
- **Operational Database** ใช้กับงานประจำวันและข้อมูลปัจจุบัน; **Data Warehouse** ใช้เก็บข้อมูลประวัติที่จัดรูปแล้วเพื่อวิเคราะห์
- **Data Lake** เก็บข้อมูล raw หลากหลายรูปแบบ ใช้แนวคิด schema-on-read คือค่อยตีความ schema ตอนนำไปใช้
- ถ้าโจทย์บอกข้อมูลสะอาด เป็นโครงสร้าง และใช้ทำ BI ระยะยาว ให้นึกถึง Data Warehouse; ถ้าบอก raw, diverse, future analytics ให้นึกถึง Data Lake
- **CAP** เตือนว่า distributed system ต้อง trade-off ระหว่าง Consistency, Availability, และ Partition Tolerance
- **Elasticity** คือเพิ่มลด resource ตาม demand ได้ เหมาะกับ cloud workload ที่ traffic ขึ้นลงไม่คงที่

## RECAP 7: Normalization แบบจับใจหทัยตาราง

- **1NF** ตรวจสอบว่าทุกช่องเป็น atomic หรือไม่ ถ้ามี list, repeating group, หลายค่าในช่องเดียว ให้แตกออกเป็นหลายแถวหรือตารางลูก
- **2NF** ตรวจสอบเฉพาะกรณี primary key เป็น composite key ถ้า non-prime attribute พึ่งแค่บางส่วน of key ให้แยกตารางออก
- **3NF** ตรวจสอบว่า non-key attribute ไปกำหนด non-key attribute ด้วยกันหรือไม่ ถ้ามีให้แยก determinant นั้นออกเป็นตารางใหม่
- เวลาทำข้อสอบให้ขีดเส้นใต้ PK ก่อน แล้ววง FK หรือเขียนกำกับไว้ จากนั้นค่อยลาก FD ว่าใครกำหนดใคร
- ถ้าโจทย์ให้ตารางที่ดูเหมือน normalize แล้ว อย่าเชื่อน้าตา ให้ถามว่า attribute นี้ขึ้นกับ key ทั้งก่อนจริงไหม และ determinant ฝั่งซ้ายเป็น key ของตารางนั้นหรือเปล่า
- Pattern หลอกบ่อย: ตารางกลางที่มี composite key แต่ยังแอบเก็บชื่อพนักงาน ชื่อโปรเจกต์ หรือชื่อแผนกไว้ด้วย ทำให้เกิด partial/transitive dependency
- ตารางหลังแก้ 3NF ควรอ่านแล้วขีดว่า table หลักเก็บ entity, table กลางเก็บ relationship, และ FK ใช้โยงกลับโดยไม่ duplicate ชื่อที่ lookup ได้